

ՀԱՅԱՍՏԱՆԻ ՀԱՆՐԱՊԵՏՈՒԹՅՈՒՆ ԿՐԹՈՒԹՅԱՆ ԵՎ ԳԻՏՈՒԹՅԱՆ
ՆԱԽԱՐԱՐՈՒԹՅՈՒՆ
ՀԱՅԱՍՏԱՆԻ ԱԶԳԱՅԻՆ ՊՈԼԻՏԵԽՆԻԿԱԿԱՆ ՀԱՄԱԼՍԱՐԱՆ
ՏՀՏԷ ԻՆՍՏԻՏՈՒՏ ՏԵՂԵԿԱՏՎՈՒԹՅԱՆ ԱՆՎՏԱՆԳՈՒԹՅԱՆ ԵՎ ԾՐԱԳՐԱՅԻՆ
ԱՊԱՀՈՎՄԱՆ ԱՄԲԻՈՆ



Լաբորատոր աշխատանք 11

Ինստիտուտ՝ ՏՀՏԷ

Առարկա՝ Ծրագրային ապահովման անվտանգություն

Թեմա՝ Ծրագրային ընթացակարգերի իրական պարամետրերի քանակի աղավաղում
Խումբ՝ SS-355

Ուսանող՝ Մելքոնյան Սոնա

Երևան 2026

ԹԵՄԱ 1. Argument Count Obfuscation

Dummy Parameters

obf_add(a, b, fake1, fake2) ֆունկցիան իրականում օգտագործում է միայն a և b պարամետրերը: fake1 և fake2 պարամետրերը ավելացված են վերլուծողին շփոթեցնելու համար: Սա reverse engineering-ի ժամանակ դժվարացնում է հասկանալ, թե որ պարամետրերն են իրականում կարևոր:

Packed Parameters

Այս մեթոդում երկու արժեք պահվում են մեկ int փոփոխականի մեջ:
packed >> 16 գործողությամբ ստացվում է a արժեքը, իսկ packed & 0xFFFF-ով՝ b արժեքը:
Այս ձևով իրական պարամետրերը թաքցվում են մեկ ընդհանուր արժեքի ներսում:

Variadic Obfuscation

obf_add(int count, ...) ֆունկցիան օգտագործում է variadic arguments (...):
Ֆունկցիան runtime-ի ժամանակ է ստանում պարամետրերը va_arg() միջոցով: Սա բարդացնում է static analysis-ը, քանի որ ֆունկցիայի իրական պարամետրերի քանակը ակնհայտ չէ:

Python Dummy + Hidden Params

obf_add(a, b, *args) ֆունկցիայում *args պարամետրերը չեն օգտագործվում: Դրանք ավելացված են obfuscation-ի համար, որպեսզի իրական պարամետրերը թաքցվեն:

Python Packed Parameters

Երկու արժեք միավորվում են մեկ integer-ի մեջ (5 << 16) | 3 գործողությամբ:
Հետո դրանք վերականգնվում են bitwise գործողություններով:

ԹԵՄԱ 2. Code Scattering in Memory

Function Pointer Scattering

part1() և part2() ֆունկցիաները կանչվում են function pointer-ների միջոցով (f1, f2):
Սա թաքցնում է execution flow-ը և բարդացնում է disassembler-ների աշխատանքը:

Dynamic Code Execution (C++)

Այս օրինակում machine code-ը պահվում է byte array-ի մեջ և runtime-ի ժամանակ տեղադրվում executable memory-ում (VirtualAlloc):

Հետո այդ հիշողությունը cast է արվում function pointer-ի և անմիջապես կատարվում:

Սա իրական runtime code execution-ի օրինակ է:

Python Function Dispatch

Ֆունկցիաները պահվում են list-ի մեջ և կանչվում index-ով:

Այս մոտեցումը թաքցնում է, թե runtime-ի ժամանակ կոնկրետ որ ֆունկցիան է աշխատելու:

Python Dynamic Code Execution

exec() ֆունկցիայի միջոցով Python code-ը ստեղծվում և կատարվում է runtime-ի ընթացքում:

Սա բարդացնում է static analysis-ը, քանի որ ամբողջ code-ը սկզբից տեսանելի չէ:

1. VM-based obfuscator

ծրագիրը չի կատարում գործողությունը ուղիղ C++ կոդով: Փոխարենը ստեղծված է custom bytecode, որտեղ հրամանները ներկայացված են թվերով՝ PUSH, ADD, PRINT, HALT: Ծրագիրը աշխատում է որպես փոքր virtual machine, որը կարդում է bytecode-ը և կատարում հրամանները: Սա բարդացնում է reverse engineering-ը, քանի որ իրական լոգիկան թաքցված է bytecode-ի մեջ:

```
bytecode.cpp > OpCode
1  #include <iostream>
2  using namespace std;
3
4  enum OpCode {
5      PUSH = 1,
6      ADD = 2,
7      PRINT = 3,
8      HALT = 4
9  };
10
11 int main() {
12     int bytecode[] = {
13         PUSH, 5,
14         PUSH, 3,
15         ADD,
16         PRINT,
17         HALT
18     };
19
20     int stack[10];
21     int sp = 0;
22     int ip = 0;
23
24     while (true) {
25         int op = bytecode[ip++];
26
27         if (op == PUSH) {
28             stack[sp++] = bytecode[ip++];
29         }
30         else if (op == ADD) {
31             int b = stack[--sp];
32             int a = stack[--sp];
33             stack[sp++] = a + b;
34         }
35         else if (op == PRINT) {
36             cout << stack[sp - 1] << endl;
37         }
38         else if (op == HALT) {
39             break;
40         }
41     }
42
43     return 0;
44 }
```

```
sona@WIN-L4E9DMLMG68:/mnt/c/Users/user/Desktop/Software Security/lab11$ ./bytecode.exe
8
```

2. Control flow flattening

ծրագրի սովորական հաջորդական հոսքը փոխարինված է state machine-ով: while և switch կառուցվածքների միջոցով ծրագիրը քայլ առ քայլ անցնում է state-երով: Արդյունքում իրական execution flow-ը դժվար է հասկանալ static analysis-ի ժամանակ:

```
control_flow_flattening.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int hidden_logic(int a, int b) {
5      int state = 0;
6      int result = 0;
7
8      while (true) {
9          switch (state) {
10             case 0:
11                 result = a + b;
12                 state = 1;
13                 break;
14
15             case 1:
16                 result = result * 2;
17                 state = 2;
18                 break;
19
20             case 2:
21                 return result;
22             }
23         }
24     }
25
26 int main() {
27     cout << hidden_logic(5, 3) << endl;
28     return 0;
29 }
```

```
sona@WIN-L4E9DMLMG68:/mnt/c/Users/user/Desktop/Software Security/lab11$ ./control_flow_flattening.exe
16
```